

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

Создание веб-приложений с использованием Angular и React

Урок №10

React:
расширенные
приемы

Содержание

И ещё о useEffect	3
Использование PropTypes	7
Использование значений по умолчанию в PropTypes.....	12
Описание объекта в PropTypes	16
Взаимодействие с внешними сервисами.....	19
Создание локального проекта React.....	31
Домашнее задание.....	51

И ещё о useEffect

Мы уже знаем, что хук `useEffect` является аналогом `componentDidMount` и `componentDidUpdate`. Хук `useEffect` вызывается в обоих случаях. А что если нам необходимо реагировать только на `componentDidMount`, а `componentDidUpdate` проигнорировать. Когда это нам может понадобиться? Например, если нам нужно произвести какую-то инициализацию единожды, когда элемент уже встроен в DOM и рендер произошёл.

Для решения этой задачи вам нужно использовать уже знакомый второй аргумент `useEffect`. В нём нужно указать пустые квадратные скобки — `[]`.

Передача `[]` означает, что мы хотим запустить функцию из первого аргумента, только один раз, фактически при `componentDidMount`. В коде это будет выглядеть так:

```
useEffect(() => {  
    код функции  
});  
, []);
```

А, как для функциональных компонент создать аналог `componentWillUnmount`?

Решение вас удивит 😊. Нужно из функции в первом аргументе `useEffect`, вернуть код, который будет вызван перед удалением компонента из DOM. Как это выглядит в коде:

```
useEffect(() => {
  код функции
  return () => {
    код для componentWillUnmount
  };
});
```

Объединим наши знания в примере. Создадим приложение, которое будет отображать текущее время. Каждую секунду мы будем обновлять значение времени.

Внешний вид нашего приложения:

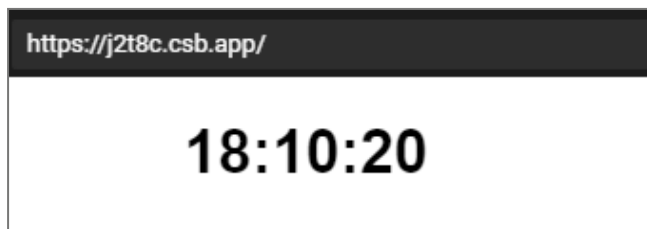


Рисунок 1

Код *App.js*:

```
import React, { useState, useEffect } from "react";
import "./styles.css";

export default function App() {
  /*
    Инициализируем состояние
  */
  const [currTime, setCurrTime] =
    useState(new Date().toLocaleTimeString());
  /*
    Функция для таймера
  */
```

```

const timerAction = () => {
  setCurrTime(new Date().toLocaleTimeString());
};

useEffect(() => {
  let handlerTimer = setInterval(timerAction, 1000);
  return () => {
    clearInterval(handlerTimer);
  };
}, []);
return (
  <div className="App">
    <h1>{currTime}</h1>
  </div>
);
}

```

Текущее время мы будем хранить в переменной состояния. Это позволяет нам возложить задачу обновления UI на React. В приложении мы используем уже известный вам из JavaScript механизм таймера. Рассмотрим код `useEffect`:

```

useEffect(() => {
  let handlerTimer = setInterval(timerAction, 1000);
  return () => {
    clearInterval(handlerTimer);
  };
}, []);

```

Таймер нам нужно создать единожды. Поэтому мы указываем во втором аргументе `[]`. Это значит, код `useEffect` будет аналогом только `componentDidMount`. Перед выгрузкой компоненты мы должны удалить таймер.

Для этого мы в `return` указали код для `componentWillUnmount`:

```
return () => {  
  clearInterval(handlerTimer);  
};
```

Мы вызываем функцию для очистки таймера. Весь остальной код примера вам должен быть уже понятен.

► Код проекта можно посмотреть по [ссылке](#).

Использование PropTypes

Мы уже умеем использовать механизм `props`. Много раз мы создавали и использовали `props` в своих собственных компонентах. Предположим вам нужно отдать свой компонент в использование другому программисту. Как описать механизм `props` вашего компонента, чтобы знакомство с ним и его использование было максимально удобным?

Вариантов много. Вы можете создать текстовый файл с описанием, разместить комментарий в коде, создать видеоролик.

Сегодня мы поговорим об ещё одном способе описания компоненты. Имя ему — `PropTypes`. Механизм `PropTypes` позволяет описать и наложить требования на `props` в коде компоненты. Для использования `PropTypes` необходимо выполнить новый импорт в коде:

```
import PropTypes from "prop-types";
```

С помощью `PropTypes` мы описываем `props` конкретного компонента. Формат использования:

```
ИМЯ_КОМПОНЕНТЫ.propTypes = {  
  Название_props1: описание,  
  Название_props2: описание,  
  ...  
};
```

Попробуем разобраться на примере. Создадим приложение, отображающее портрет писателя, его имя и фа-

милия, а также известную цитату. Внешне приложение будет выглядеть так:

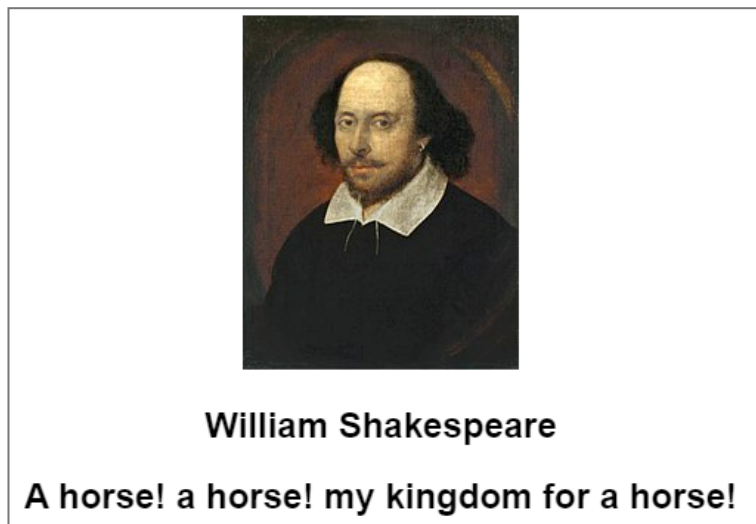


Рисунок 2

За отображение информации об авторе у нас будет отвечать компонента [Author](#). При создании этого приложения выполним разнесение компонент по разным файлам.

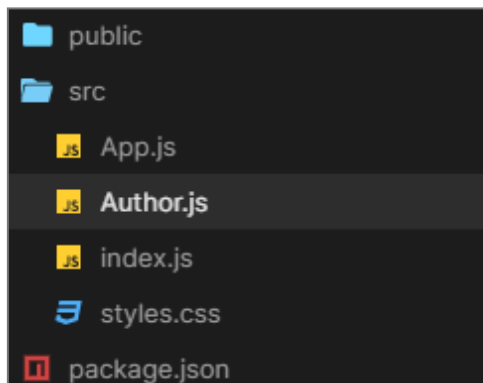


Рисунок 3

Компонента **Author** находится в отдельном файле. По традиции начнем с кода в *App.js*:

```
import React from "react";
import Author from "../Author.js";
import "../styles.css";
export default function App() {
  const path =
    "https://upload.wikimedia.org/wikipedia/commons/" +
    "thumb/a/a2/Shakespeare.jpg/187px-Shakespeare.jpg";
  const quote = "A horse! a horse! my kingdom for a horse!";
  const authorName = "William Shakespeare";
  return (
    <div className="App">
      <Author picture={path}
              name={authorName}
              quote={quote} />
    </div>
  );
}
```

В коде мы создаём компоненту **Author** и передаём ей три параметра.

Вверху файла мы подключили компоненту:

```
import Author from "../Author.js";
```

Код *Author.js*:

```
import React from "react";
import PropTypes from "prop-types";
export default function Author(props) {
  return (
    <div>
      <img src={props.picture} alt="portrait" />
    </div>
  );
}
```

```

        <h2>{props.name}</h2>
        <h2>{props.quote}</h2>
    </div>
  );
}

Author.propTypes = {
  picture: PropTypes.string.isRequired,
  name: PropTypes.string.isRequired,
  quote: PropTypes.string.isRequired
};

```

Мы импортировали `PropTypes` в начале файла.

```
import PropTypes from "prop-types";
```

После описания компоненты мы настроили `PropTypes`:

```

Author.propTypes = {
  picture: PropTypes.string.isRequired,
  name: PropTypes.string.isRequired,
  quote: PropTypes.string.isRequired
};

```

Для настройки конкретного свойства мы используем такой формат:

```
picture: PropTypes.string.isRequired,
```

В нашем примере `picture` это имя свойства, `PropTypes.string.isRequired` — описание. Мы указали, что `picture` это обязательное строковое свойство. Такие же требования предъявляются к `name` и `quote`.

Что же произойдет, если указанные пожелания будут нарушены?

Изменим код в *App.js*, чтобы спровоцировать проблему:

```
export default function App() {  
  const path =  
    "https://upload.wikimedia.org/wikipedia/commons/" +  
    "thumb/a/a2/Shakespeare.jpg/187px-Shakespeare.jpg";  
  const quote = 5;  
  const authorName = "William Shakespeare";  
  return (  
    <div className="App">  
      <Author picture={path}  
        name={authorName}  
        quote={quote} />  
    </div>  
  );  
}
```

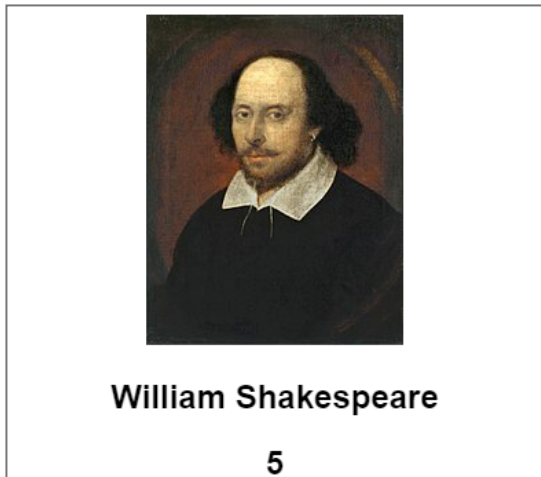


Рисунок 4

Теперь `quote` нарушает наши требования. Мы требуем строку, а передаётся число. Тем не менее приложение запускается, как и раньше (рис. 4).

Почему так происходит? Дело в том, что результат проверки требований **PropTypes** отображается в консоли и никак не влияет на работу приложения (рис. 5).

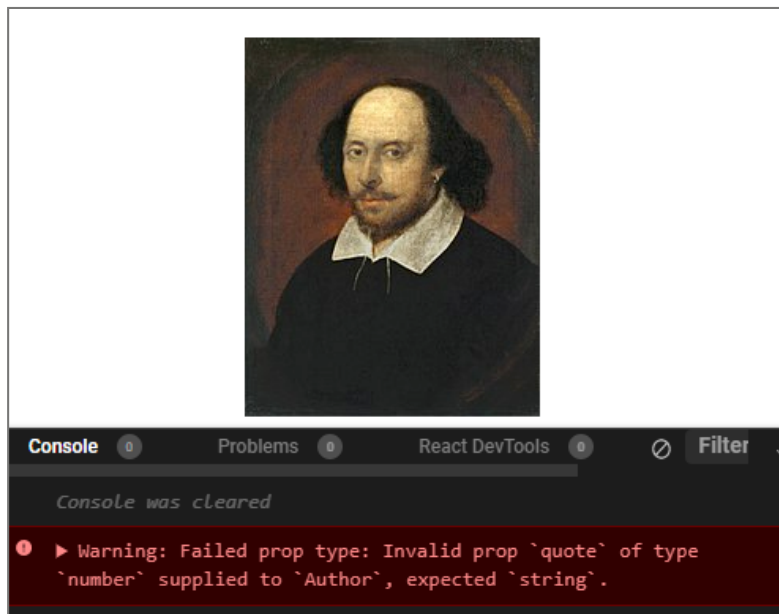


Рисунок 5

Именно поэтому, не забывайте проверять, что указано в консоли, когда запускаете приложения.

► Код проекта можно посмотреть по [ссылке](#).

Использование значений по умолчанию в **PropTypes**

Пользователь вашей компоненты по той или иной причине может не передать значение для свойства. Если для вашего кода это может быть критично, вам нужно предусмотреть значение по умолчанию. Когда значение

для свойства не будет указано явно, React возьмёт значение по умолчанию.

Синтаксис определения значений по умолчанию:

```
ИМЯ_КОМПОНЕНТЫ.defaultProps = {  
  Имя1_props: значение,  
  .....  
};
```

Создадим пример использования значений по умолчанию. Внешний вид нашего примера:

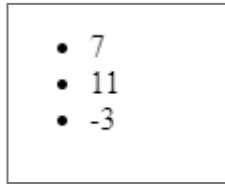


Рисунок 6

Мы отображаем список числовых значений. Разнесем наш код по разным файлам.

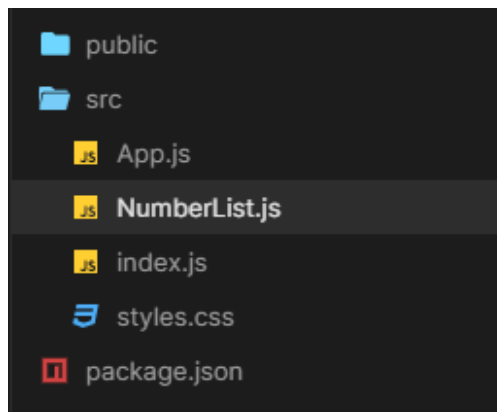


Рисунок 7

Компонента `NumberList` отвечает за показ списка чисел.
Код файла *App.js*:

```
import React from "react";
import NumberList from "../NumberList.js";

import "../styles.css";
export default function App() {
  const arr = [7, 11, -3];
  return (
    <>
      <NumberList values={arr} />
    </>
  );
}
```

Мы передали массив `arr` в качестве значения для свойства `values` компоненты `NumberList`.

Код файла *NumberList.js*:

```
import React from "react";
import PropTypes from "prop-types";

export default function NumberList(props) {
  return (
    <ul>
      {props.values.map(item => (
        <li>{item}</li>
      ))}
    </ul>
  );
}

NumberList.propTypes = {
  values: PropTypes.arrayOf(PropTypes.number).isRequired
};
```

```
NumberList.defaultProps = {  
  values: [0, 0, 0]  
};
```

В коде мы настраиваем **PropTypes**

```
NumberList.propTypes = {  
  values: PropTypes.arrayOf(PropTypes.number).  
   isRequired  
};
```

Мы указали, что свойство **values** является обязательным и в нём должен содержаться массив чисел.

```
NumberList.defaultProps = {  
  values: [0, 0, 0]  
};
```

Если данные для массива не будут переданы, в качестве значения по умолчанию React возьмет массив с тремя нулями внутри.

Добьёмся этого поведения. Для этого изменим код в *App.js*:

```
export default function App() {  
  return (  
    <>  
      <NumberList />  
    </>  
  );  
}
```

Вот как выглядит UI нашего приложения теперь:

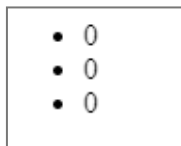


Рисунок 8

React использовал значения по умолчанию. А что будет, если мы не укажем и значения по умолчанию? Удалим строку с `defaultProps` и посмотрим, как там наш UI:



Рисунок 9

Приложение не запускается, так как свойство не было передано. Генерируется ошибка при попытке вызова `map`. Вернем наш код в рабочее состояние.

▶ Код проекта можно посмотреть по [ссылке](#).

Описание объекта в `PropTypes`

В нашем последнем примере, посвященном `PropTypes` мы рассмотрим взаимодействие с объектом. Внешний вид нашего приложения:

Honda Accord
2008

Рисунок 10

Файлы нашего приложения:

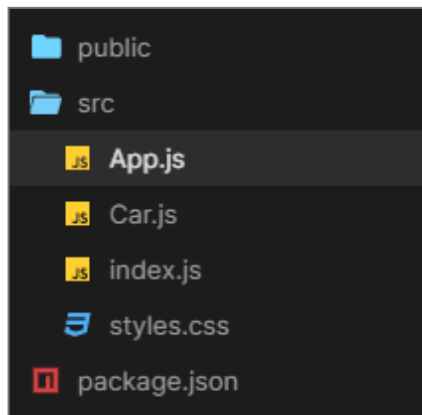


Рисунок 11

Car.js содержит код компоненты автомобиля. Код файла *App.js*:

```
import React from "react";
import Car from "../Car.js";
import "../styles.css";

export default function App() {
  const obj = {
    name: "Honda Accord",
    year: 2008
  };
}
```

```

return (
  <div className="App">
    <Car details={obj} />
  </div>
);
}

```

Мы создали объект `obj`, наполнили его, и передали в качестве значения для свойства `details`. Рассмотрим, как описан `PropTypes` внутри `Car.js`:

```

import React from "react";
import PropTypes from "prop-types";
export default function Car(props) {
  return (
    <div>
      <h2>{props.details.name}</h2>
      <h2>{props.details.year}</h2>
    </div>
  );
}

Car.propTypes = {
  details: PropTypes.exact({
    name: PropTypes.string.isRequired,
    year: PropTypes.number.isRequired
  }).isRequired
};

```

Для описания требований к объекту мы использовали вызов метода `exact`:

```

Car.propTypes = {
  details: PropTypes.exact({
    name: PropTypes.string.isRequired,

```

```
    year: PropTypes.number.isRequired  
  }).isRequired  
};
```

У нашего объекта должно быть два свойства: `name` — обязательное строковое значение, `year` — обязательное числовое значение. Кроме того, само свойство `details` является обязательным.

► Код проекта можно посмотреть по [ссылке](#).

Взаимодействие с внешними сервисами

При создании веб-приложений часто возникает необходимость взаимодействовать с внешними сервисами для получения данных. Поговорим о том, как можно настроить этот механизм.

Путей решения проблемы много. Мы будем в наших примерах использовать `axios` — `http` клиент для взаимодействия с внешними источниками.

Для подключения `axios` к проекту нужно кликнуть в левом нижнем углу на кнопку `Add dependency`.

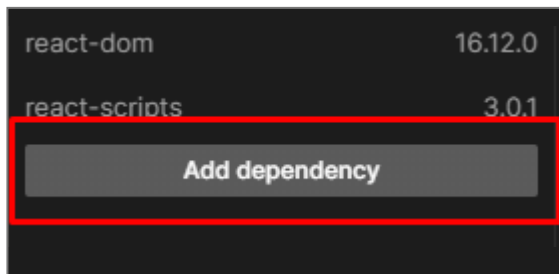


Рисунок 12

В новом окне нужно вписать название [axios](#):

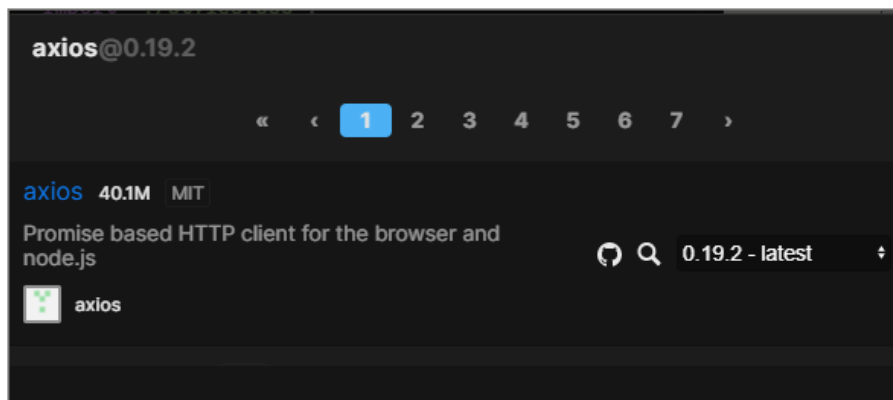


Рисунок 13

И кликнуть по названию http-клиента. После этого все возможности [axios](#) доступны в вашем проекте.

Рассмотрим принципы работы [axios](#) на примере. Создадим приложение, которое будет отображать информацию о пользователе GitHub. Начальный вид нашего приложения:

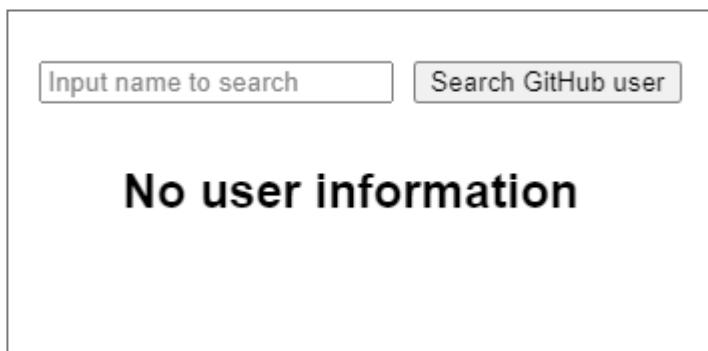


Рисунок 14

Результат поиска пользователя с ником [andrew](#):

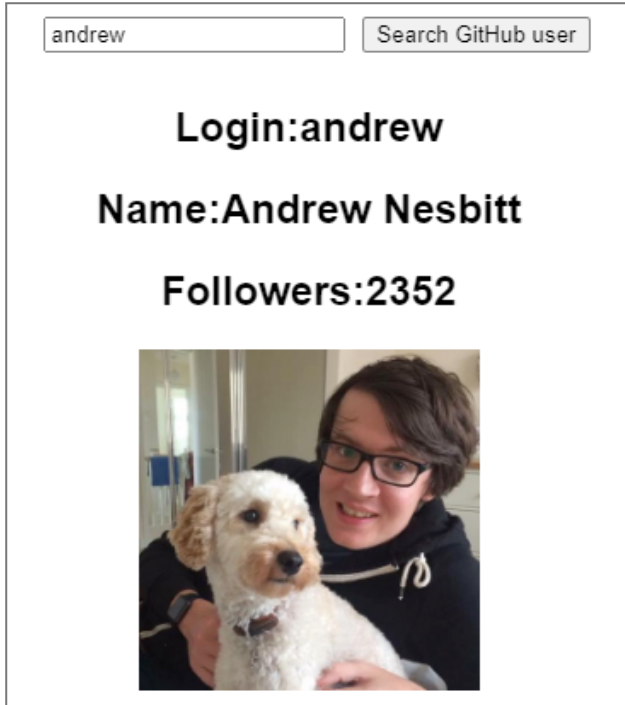


Рисунок 15

Код *App.js*:

```
import React, { useState } from "react";
import axios from "axios";
import "./styles.css";

function SearchForm(props) {
  /*
    Инициализируем состояние для текстового поля
  */
  const [userName, setUserName] = useState("");
  const handlerChange = event => {
    setUserName(event.target.value);
  };
}
```

```

/*
  В обработчике Submit посылаем запрос на данные
  пользователя. После получения данных вызываем
  функцию из компонента App для обновления состояния
*/
const handlerSubmit = async event => {
  event.preventDefault();
  const query = 'https://api.github.com/users/
                ${userName}';
  let resp = await axios.get(query);
  /*
    Вызываем функцию из родителя. Для нас это
    компонент App
  */
  props.onSubmit(resp.data);
};

return (
  <form onSubmit={handlerSubmit}>
    <input
      type="text"
      style={{ margin: "10px" }}
      required
      placeholder="Input name to search"
      value={userName}
      onChange={handlerChange}
    />
    <input type="submit" value="Search GitHub user" />
  </form>
);
}

function UserInfo(props) {
  if (typeof props.login === "undefined")
    return <h2>No user information</h2>;
  return (
    <div>
      <h2>Login:{props.login}</h2>
      <h2>Name:{props.name}</h2>
    </div>
  );
}

```

```

    <h2>Followers:{props.followers}</h2>
    <img src={props.avatar_url}
        alt="avatar" width="200" />
  </div>
);
}
export default function App() {
  /*
    Инициализируем состояния для пользователя
    При старте приложения это пустой объект
  */
  const [userObject, setUserObject] = useState({});
  /* Обновляем состояние полученными данными */
  const updateUser = user => {
    setUserObject(user);
  };
  return (
    <div className="App">
      <SearchForm onSubmit={updateUser} />
      <UserInfo {...userObject} />
    </div>
  );
}

```

Напомним, что для использования [axios](#) нужно подключить его в проект. Кроме того, требуется импортировать его для использования в конкретном файле:

```
import axios from "axios";
```

Большая часть кода не должна вызывать у вас сложностей. Всё это мы рассматривали ранее. Нам нужно поговорить о коде для получения данных о GitHub пользователей.

```
const handlerSubmit = async event => {
  event.preventDefault();

```

```
const query = 'https://api.github.com/users/
               ${userName}';
let resp = await axios.get(query);
/*
    Вызываем функцию из родителя
    Для нас это компонент App
*/
props.onSubmit(resp.data);
};
```

В обработчике нажатия на кнопку **Submit** мы делаем запрос к API GitHub. Для отсылки запроса нужно вызвать метод **get**. Мы используем конечную точку GitHub для получения информации о конкретном пользователе. Путь к этой точке выглядит так:

```
https://api.github.com/users/имя_пользователя
```

В ответ на запрос к этой точке GitHub возвращает информацию о пользователе. Мы хотим получить эту информацию асинхронно. Это значит, что после отсылки запроса наш скрипт не будет ожидать получение ответа, вместо этого он продолжит своё исполнение. Когда ответ будет получен, выполниться часть кода, указанная после строки, отсылающей запрос.

Для того, чтобы запрос был асинхронным нужно указать **async** перед функцией содержащей асинхронный запрос. Кроме того нужно указать **await** в строке асинхронного запроса.

```
const handlerSubmit = async event => {
    event.preventDefault();
```


Указали `async` перед телом обработчиком.

```
const query = 'https://api.github.com/users/  
    ${userName}';  
let resp = await axios.get(query);
```

Указали `await` перед отсылкой запроса через `get`. Когда ответ будет получен, скрипт вернется внутрь обработчика `Submit` и запишет результат запроса в `resp`.

Что же содержится внутри `resp`?

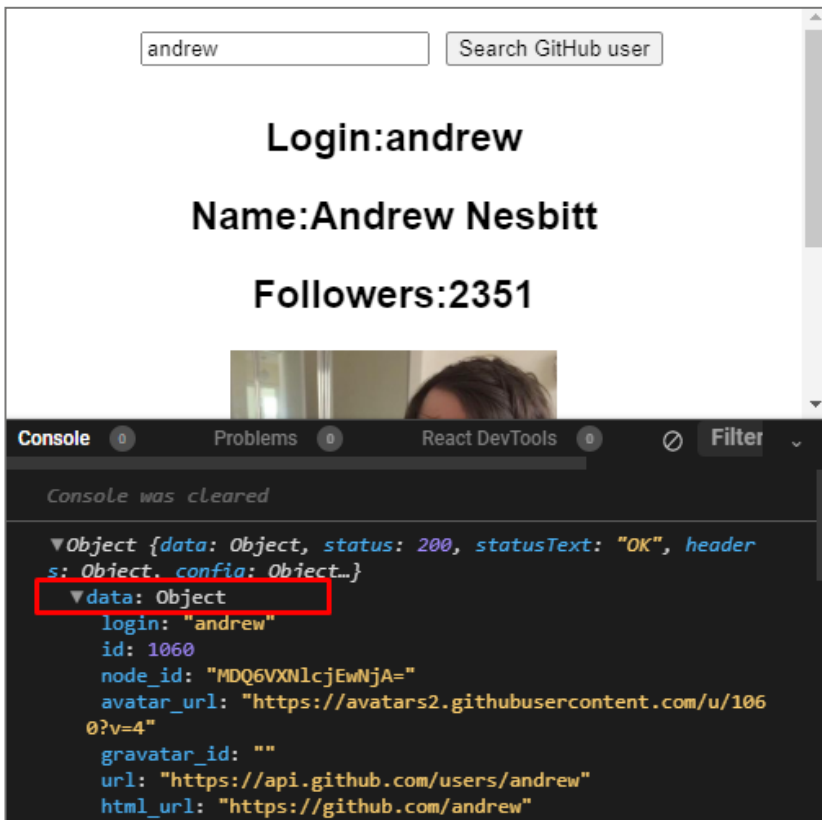


Рисунок 16

Внутри ответа `resp` содержится свойство `data`, в котором будут данные об искомом пользователе. Полученные данные наше приложение отображает внутри UI-интерфейса нашего приложения.

При работе с внешними источниками очень важно знать, как обращаться с API конкретного ресурса, поэтому разработку взаимодействия с внешним ресурсом надо начинать с чтения его документации. Например, в нашем случае это API GitHub.

► Код проекта можно посмотреть по [ссылке](#).

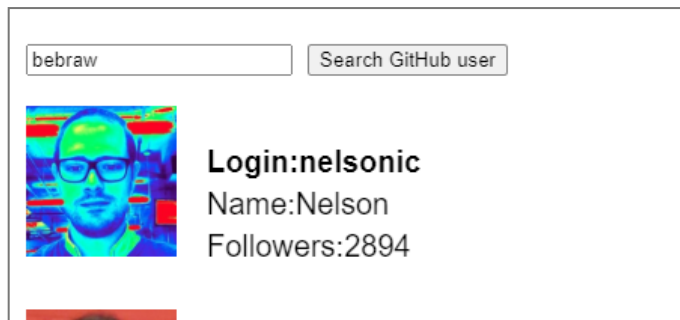
Немного модифицируем наш пример. Все найденные пользователи будут добавляться в список. Начальный вид нашего приложения:



The image shows a simple web interface with a rectangular container. Inside, there is a text input field on the left with the placeholder text "Input name to search" and a button on the right labeled "Search GitHub user".

Рисунок 17

Внешний вид нашего приложения после нескольких поисков на рисунке 18.



The image shows the application interface after a search. The input field now contains the text "bebraw". The button remains "Search GitHub user". Below the input field, there is a profile picture of a man with glasses. To the right of the picture, the following text is displayed: "Login:nelsonic", "Name:Nelson", and "Followers:2894".

Рисунок 18

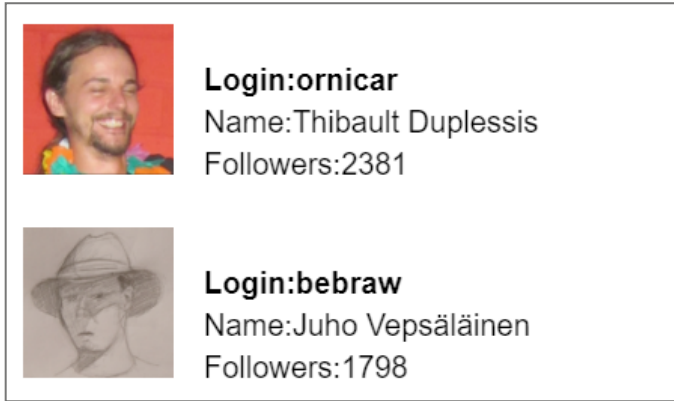


Рисунок 18 (продолжение)

В коде этого приложения мы распределили компоненты по нескольким файлам.

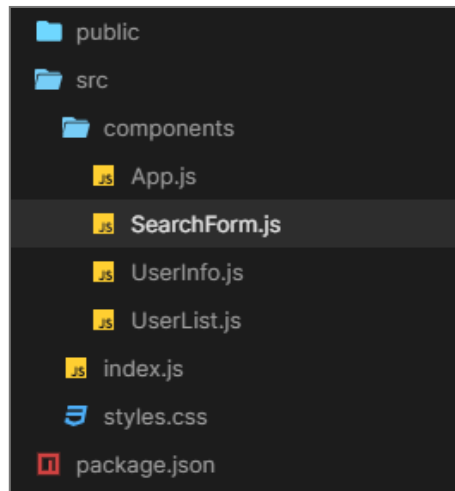


Рисунок 19

SearchForm.js содержит поисковую компоненту. *UserInfo.js* содержит информацию о конкретном пользователе. *UserList.js* содержит список всех найденных пользователей.

Начнем с кода в *App.js*:

```
import React, { useState } from "react";
import SearchForm from "../SearchForm";
import UserList from "../UserList";

import "../styles.css";

export default function App() {
  /*
    Инициализируем состояния для пользователя
    При старте приложения это пустой объект
  */

  const [usersArr, setUsersArr] = useState([]);
  /*
    Обновляем состояние полученными данными
  */
  const updateUser = user => {
    setUsersArr([...usersArr, user]);
  };

  return (
    <>
      <SearchForm onSubmit={updateUser} />
      <UserList users={usersArr} />
    </>
  );
}
```

Мы храним переменную состояния (массив пользователей) внутри компоненты **App**. Такое решение было принято, так как этот массив нужен разным компонентам. Фактически **App** это наш родительский компонент, а остальные компоненты дочерние.

Код *SearchForm.js*:

```

import React, { useState } from "react";
import axios from "axios";

export default function SearchForm(props) {
  /*
    Инициализируем состояние для текстового поля
  */
  const [userName, setUserName] = useState("");
  const handlerChange = event => {
    setUserName(event.target.value);
  };
  /*
    В обработчике Submit посылаем запрос на данные
    пользователя. После получения данных вызываем
    функцию из компонента App для обновления
    состояния
  */
  const handlerSubmit = async event => {
    event.preventDefault();
    const query = 'https://api.github.com/users/'
                  `${userName}`;
    let resp = await axios.get(query);
    /*
      Вызываем функцию из родителя
      Для нас это компонент App
    */
    props.onSubmit(resp.data);
  };

  return (
    <form onSubmit={handlerSubmit}>
      <input
        type="text"
        style={{ margin: "10px" }}
        required

```

```

        placeholder="Input name to search"
        value={userName}
        onChange={handlerChange}
      />
      <input type="submit" value="Search GitHub user" />
    </form>
  );
}

```

Этот код вы видели в прошлом примере. Код *UserInfo.js*:

```

import React from "react";

export default function UserInfo(props) {
  return (
    <div className="github-user">
      <img src={props.avatar_url}
        alt="avatar"
        className="avatar" />
      <div className="main">
        <div className="login">
          Login:{props.login}
        </div>
        <div className="other">
          Name:{props.name}
        </div>
        <div className="other">
          Followers:{props.followers}
        </div>
      </div>
    </div>
  );
}

```

Очень простой компонент, который отвечает за отображение конкретного пользователя.

Код *UserList.js*:

```
import React from "react";
import UserInfo from "../UserInfo";

export default function UserList(props) {
  return (
    <div>
      {props.users.map(user => (
        <UserInfo key={user.id} {...user} />
      ))}
    </div>
  );
}
```

Этот компонент отображает список пользователей с помощью уже известной вам техники. Внимательно проанализируйте код этого примера, пройдите все его сложные места, попробуйте восстановить его с нуля. Мы уверены, что вы с успехом справитесь с этим заданием!

► Код проекта можно посмотреть по [ссылке](#).

Создание локального проекта React

Вот и наступил знаменательный момент! Сейчас мы научимся создавать проект React-приложения на вашем компьютере. Это не означает, что нам нужно забыть всё, что мы использовали в CodeSandbox. К уже полученным знаниям мы добавим новые 😊.

Начнем с программного обеспечения, которое требуется установить.

Первый шаг — Visual Studio Code. Это популярный, бесплатный редактор исходного кода от компании Microsoft.

Большое количество разработчиков используют его каждый день. VS Code обладает хорошей скоростью работы. В VS Code встроена возможность подключения расширений (*extensions*). Это позволяет существенно расширить базовые возможности редактора.

Для загрузки VS Code зайдите по адресу <https://code.visualstudio.com/>.

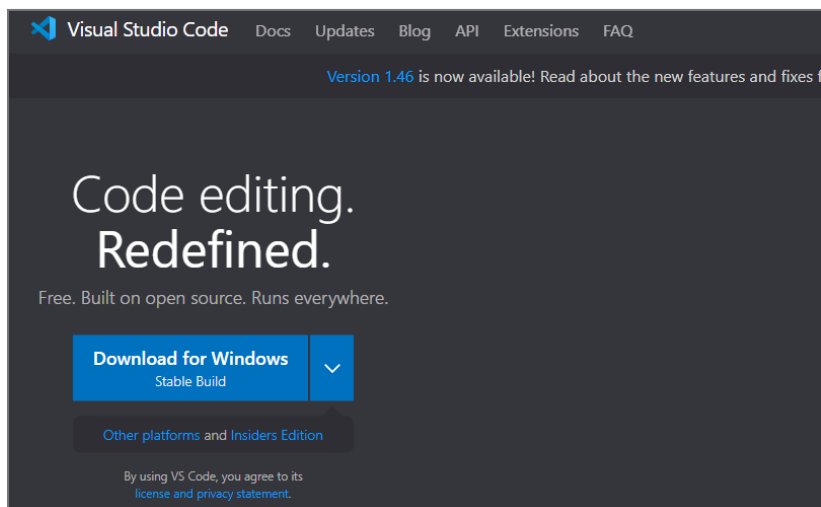


Рисунок 20

Второй шаг — установка *Node.js*. *Node.js* — это JavaScript движок, построенный на движке Chrome V8 JavaScript engine. *Node.js* создан для построения масштабных сетевых приложений. В рамках нашего курса мы будем использовать *npm* — менеджер пакетов, входящий в состав *Node.js*.

Что такое пакет? Это контейнер, содержащий набор файлов, директорий и т.д. Каждый пакет предназначен для решения конкретной задачи. Настройки конкретного пакета описываются в файле *package.json*. Менеджер

пакетов **npm** позволяет программисту использовать уже существующие пакеты.

Для загрузки *Node.js* зайдите по адресу <https://nodejs.org/>.

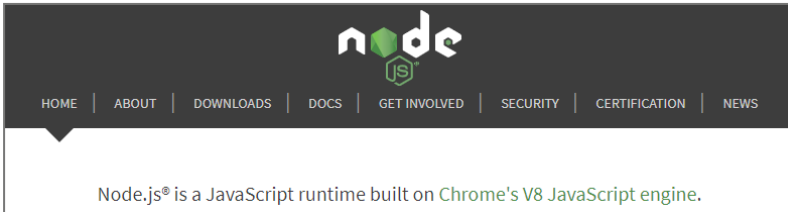


Рисунок 21

После установки VS Code и *Node.js* мы почти готовы создать первое React-приложение. Однако, не будем торопиться и настроим VS Code для более продуктивной работы.

Запустим VS Code и установим несколько полезных расширений.

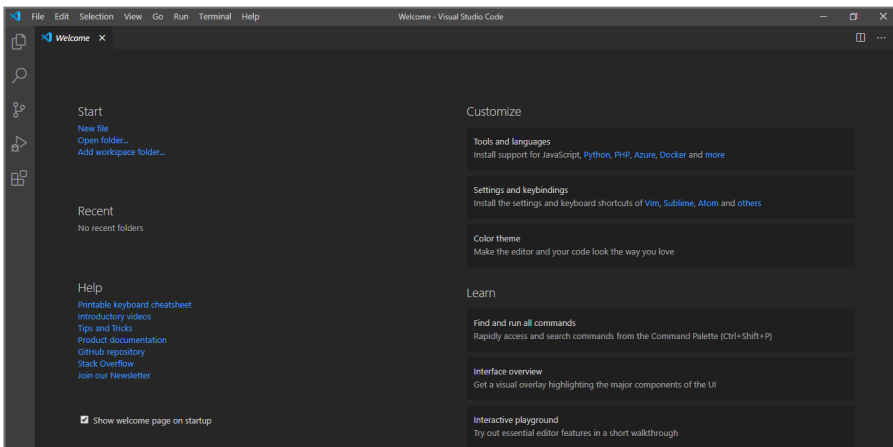


Рисунок 22

Для установки расширений нажмите на клавиатуре **Ctrl+Shift+X** или на последнюю иконку слева:

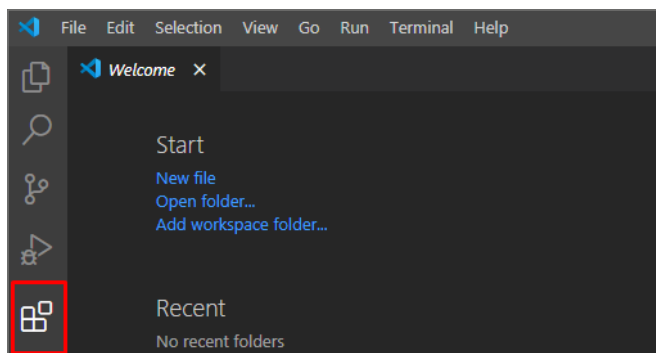


Рисунок 23

Вам откроется окно для установки расширений.

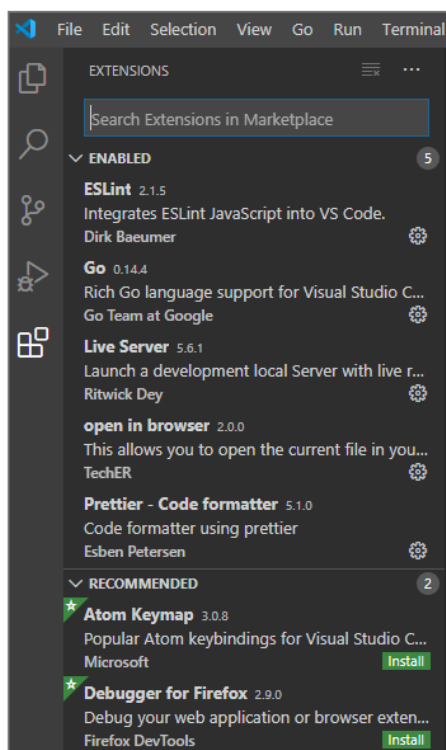


Рисунок 24

Начнем с установки расширения ESLint.

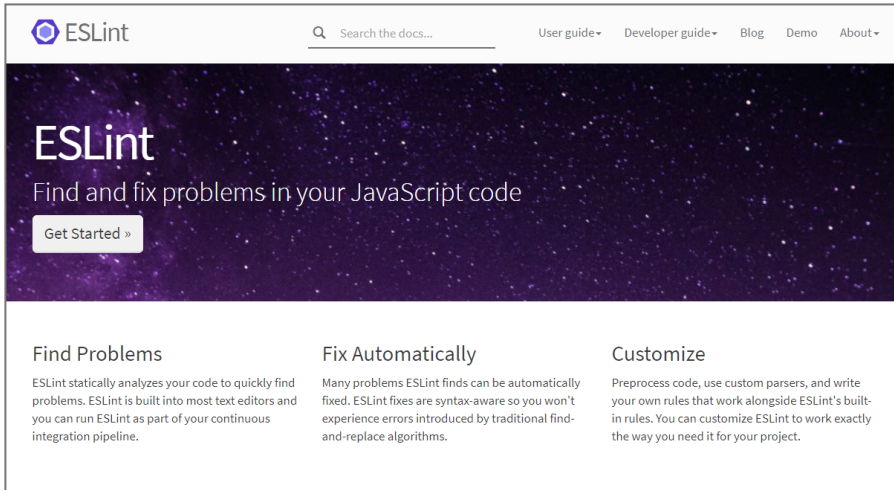


Рисунок 25

Этот инструмент анализирует ваш исходный код в поисках потенциальных ошибок. Если ошибки найдены, ESLint сообщит вам о них внутри VS Code. ESLint доступен, как отдельный инструмент, так и в виде расширения для VS Code.

Указываем ESLint в строке поиска расширений и устанавливаем его.

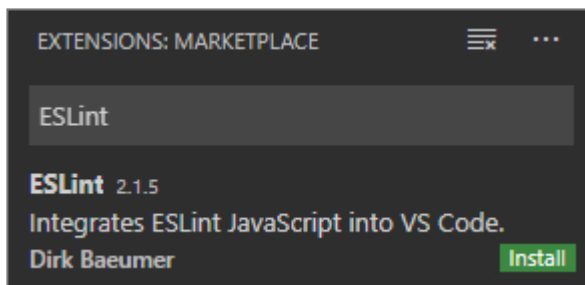


Рисунок 26

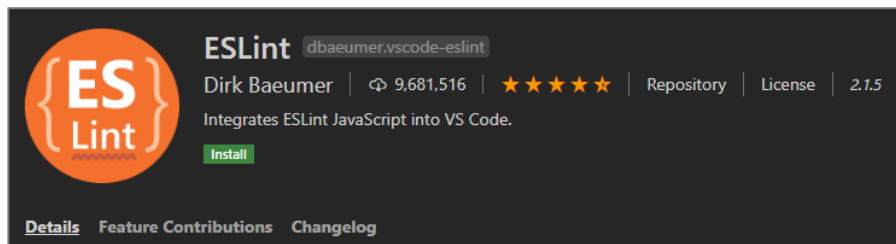


Рисунок 27

Следующее расширение, которое мы поставим — **Prettier**. Оно позволяет автоматически форматировать наш исходный код. В результате его использования у нас будет красивый, удобочитаемый код.

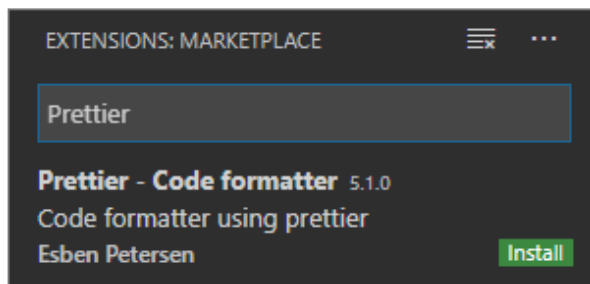


Рисунок 28

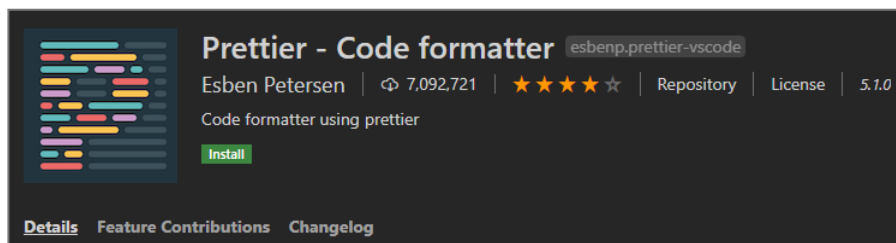


Рисунок 29

Для того, чтобы нам было удобно использовать **Prettier** настроим Visual Studio Code.

Выберем **File-> Preferences-> Settings**

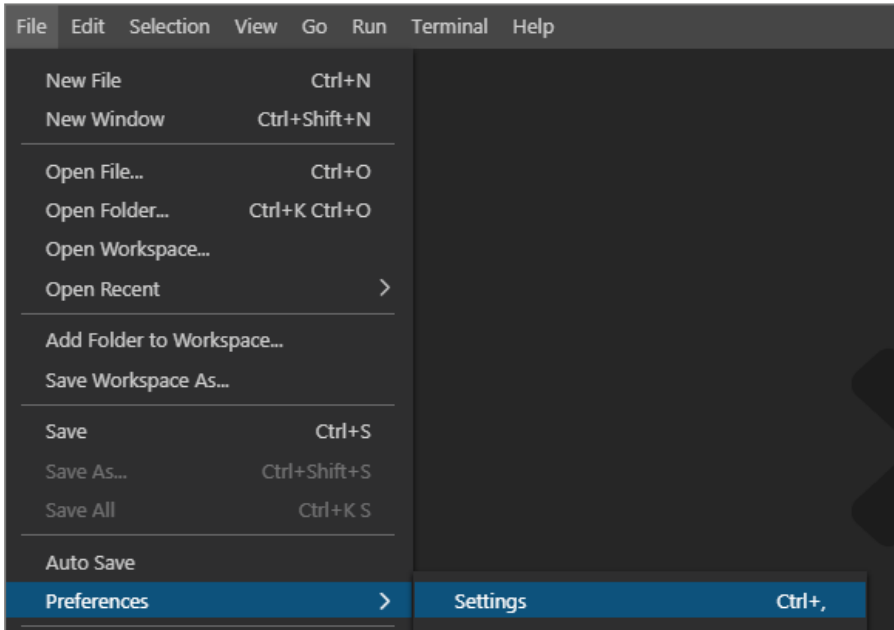


Рисунок 30

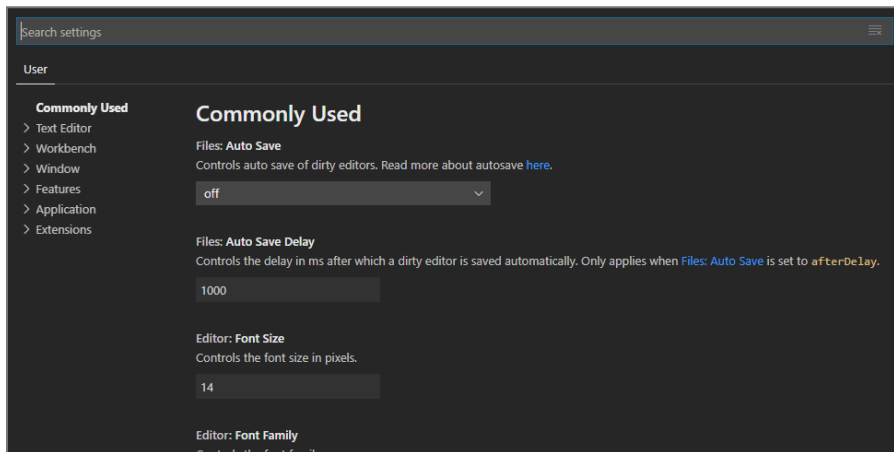


Рисунок 31

Наберите в строке поиска **format on save**

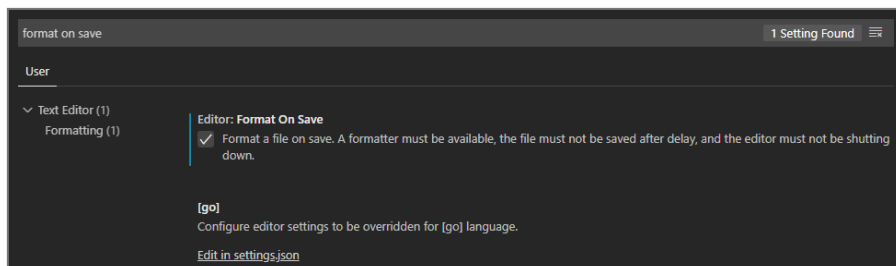


Рисунок 32

Либо выберите **Text Editor-> Formatting**

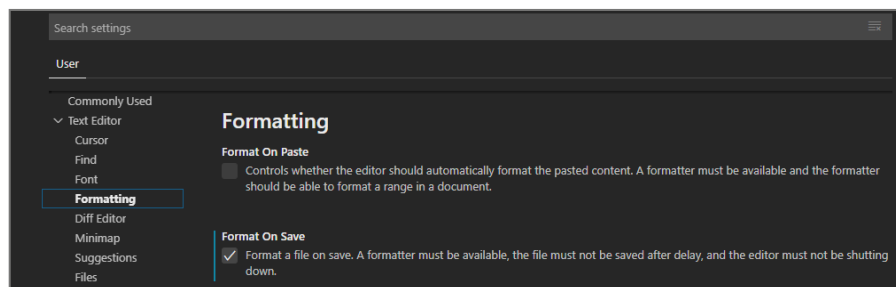


Рисунок 33

Поставьте галочку в пункте **Format a file on save**

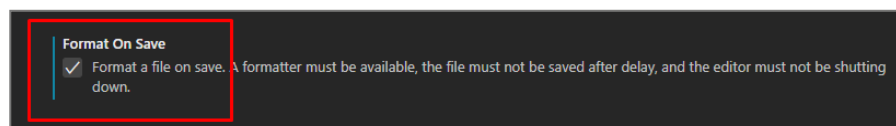


Рисунок 34

Теперь при сохранении файла **Prettier** будет автоматически форматировать код.

Мы никак не настраивали **ESLint** и **Prettier**. Нас устраивает их поведение по умолчанию. Однако, если у вас возникнет необходимость вы всегда сможете это сделать.

VS Code готов к работе. Как же нам создать каркас базового React-приложения? Вас возможно это удивит, но существует большое количество шаблонов для стартового приложения React.

Да-да 😊.

Вы можете ознакомиться с ними, например на сайте [JavaScript Stuff](#).

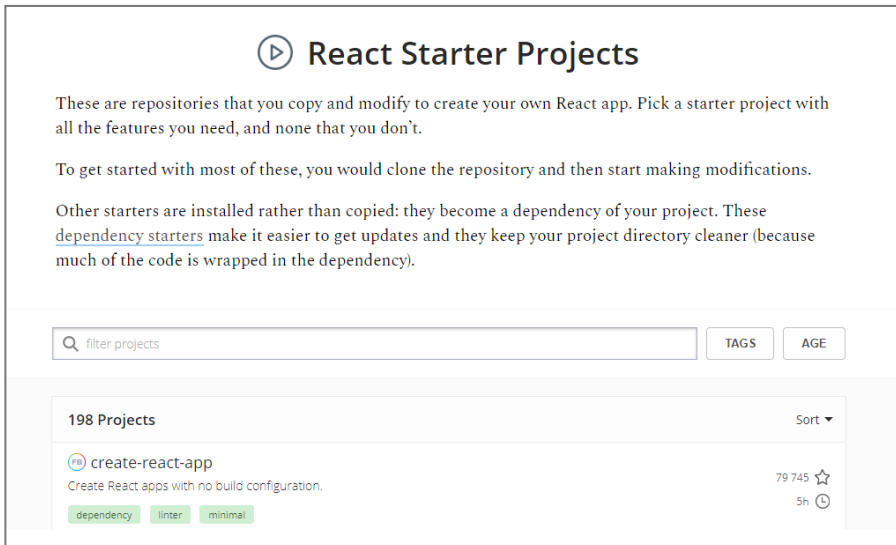


Рисунок 35

Мы выберем стартовое приложение от Facebook. Выбор сделан по причине того, что React — это продукт Facebook. И кому, как не Facebook знать, что должно входить в состав базового приложения. Название этого проекта [create-react-app](#).

Перед тем, как мы запустим создание базового приложения, поговорим об инструментах, которые используются внутри этого приложения.

Babel — JavaScript компилятор, который преобразует современный JavaScript код под любые браузеры (в том числе и очень старые). Благодаря ему мы сможем применять самые современные конструкции JavaScript не задумываясь о тонкостях реализации под ту или иную версию браузера.

► Сайт проекта: <https://babeljs.io/>.



Рисунок 36

Webpack — инструмент для сборки React-приложения. Он собирает воедино все необходимые файлы и ресурсы.

► Сайт проекта: <https://webpack.js.org/>.

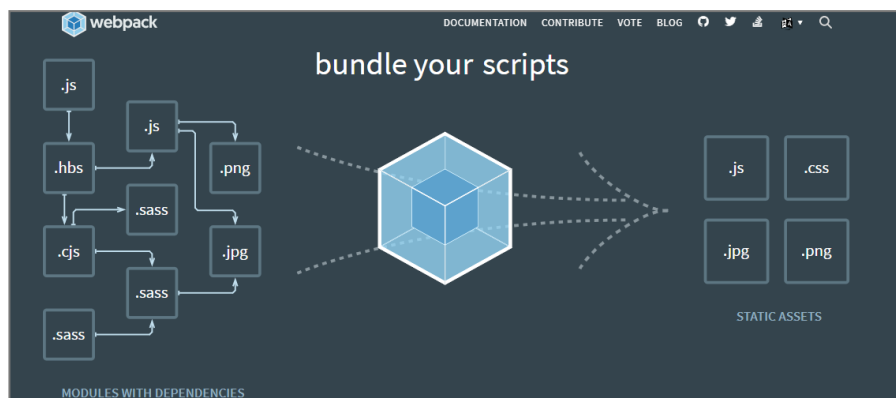


Рисунок 37

Нам не придется заниматься дополнительной настройкой этих инструментов, так как нас устраивают их настройки по умолчанию. Однако, вы всегда можете настроить тот или иной инструмент вручную.

Как вы понимаете, [CodeSandbox](#) тоже используют эти инструменты. Барабанная дробь ... Приступаем к созданию первого приложения.

Запускаем командную строку *Node.js*

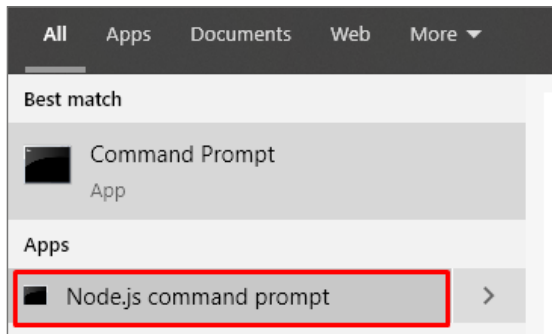


Рисунок 38

Перейдите в папку, где вы хотите создать своё React-приложение.

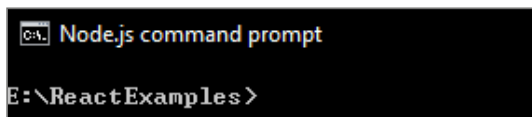


Рисунок 39

Введите команду:



Рисунок 40

- `create-react-app` — название базового приложения от Facebook;
- `hello-react` — имя нашего будущего приложения.

После запуска команды будет создано приложение *hello-react*, которое будет являться копией *create-react-app*.

Процесс создания займет некоторое время. Вам нужно будет немного подождать.



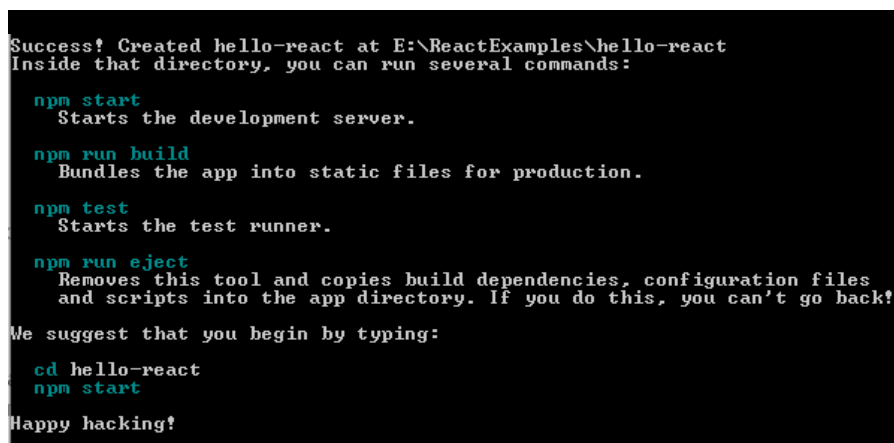
```

C:\> npm

E:\ReactExamples>npm create-react-app hello-react
[#####.....] ! extract:minimatch: sill extract rimraf@2.7.1
  
```

Рисунок 41

Когда ваше приложение будет создано вы увидите похожий вывод в консоль:



```

Success! Created hello-react at E:\ReactExamples\hello-react
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

  cd hello-react
  npm start

Happy hacking!
  
```

Рисунок 42

В вашей папке для проектов будет создана папка React-приложения:

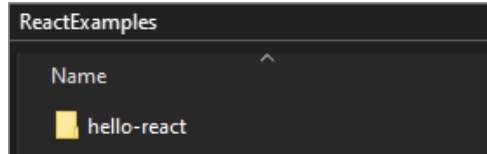


Рисунок 43

Внутри папки:

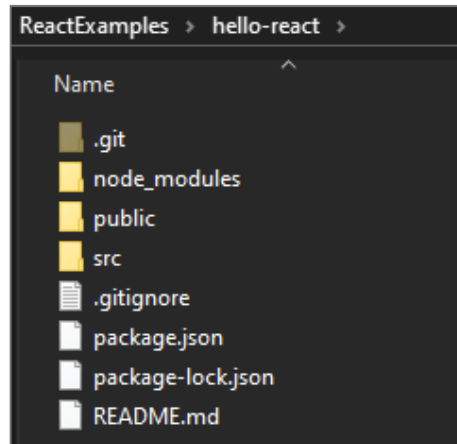


Рисунок 44

Откроем папку нашего проекта в VS Code.

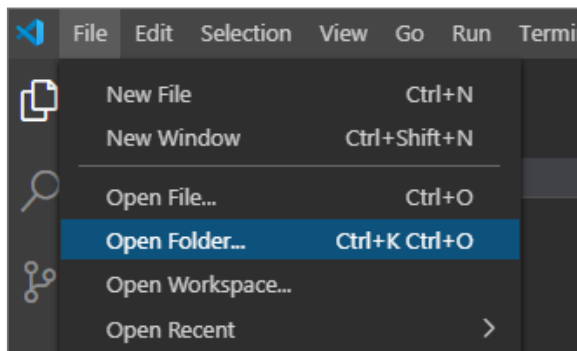


Рисунок 45

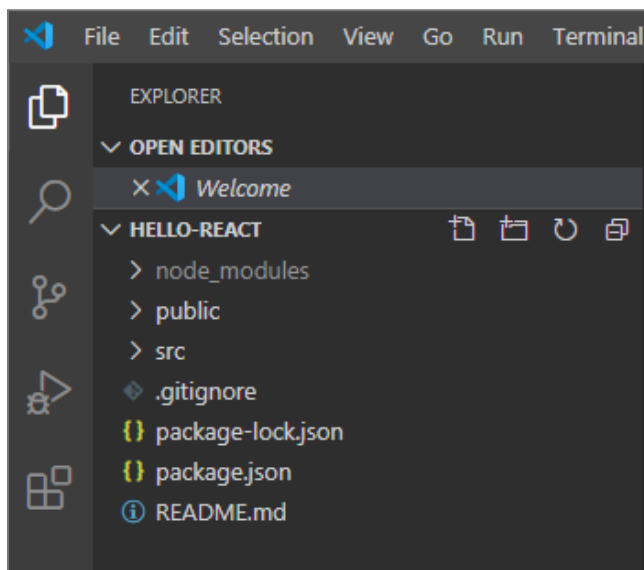


Рисунок 46

Базовый проект приложения содержит много папок и файлов. Большую часть файлов можно будет чуть позднее смело удалить.

Сейчас запустим наше приложение. Для этого откроем окно терминала.

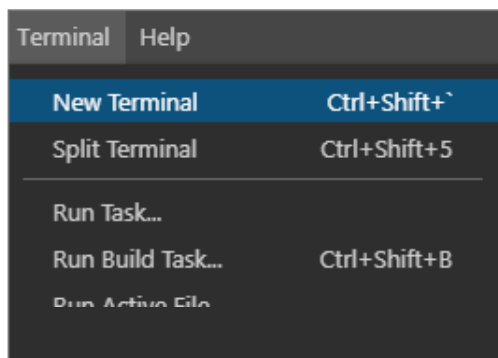
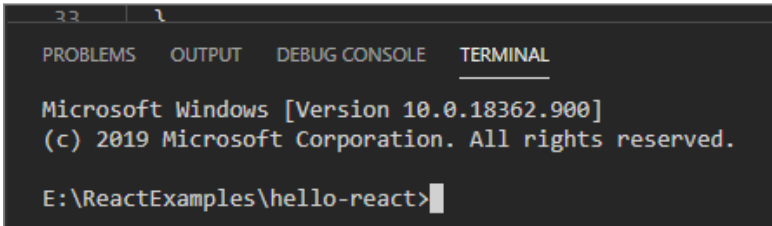


Рисунок 47

Окно терминала откроется в нижней части VS Code.



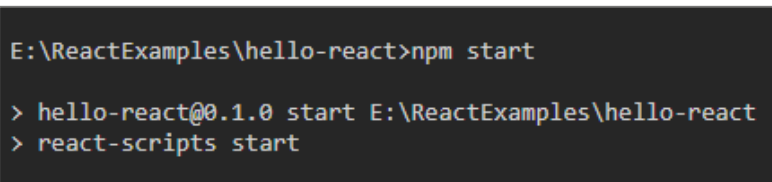
```
Microsoft Windows [Version 10.0.18362.900]
(c) 2019 Microsoft Corporation. All rights reserved.

E:\ReactExamples\hello-react>
```

Рисунок 48

Активной является корневая папка нашего приложения. Для старта приложения укажем в адресной строке:

npm start

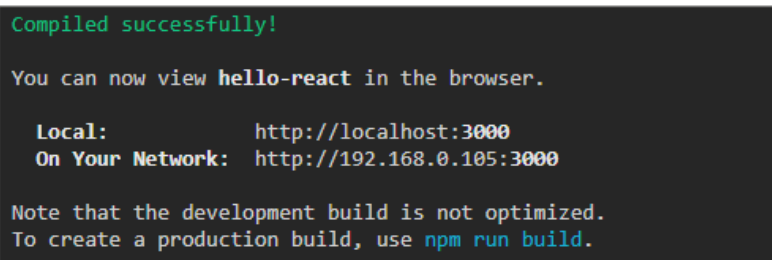


```
E:\ReactExamples\hello-react>npm start

> hello-react@0.1.0 start E:\ReactExamples\hello-react
> react-scripts start
```

Рисунок 49

Компиляция и старт приложения займет некоторое время. Когда всё будет завершено успешно, вы увидите информацию об этом в консоли. В окне указан адрес для доступа к нашему приложению.



```
Compiled successfully!

You can now view hello-react in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.0.105:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```

Рисунок 50

При старте приложения у вас будет запущен браузер с нашим приложением. Вы можете также самостоятельно запустить браузер и указать путь *http://localhost:3000* для доступа к приложению.

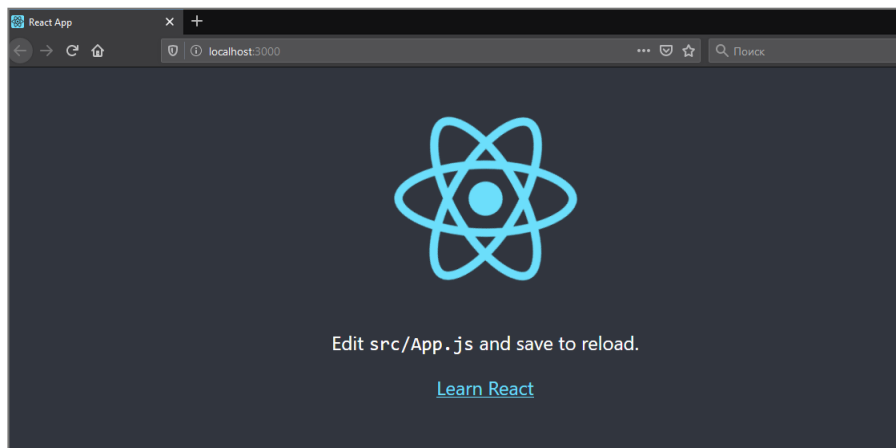


Рисунок 51

Внешний вид нашего приложения при успешной компиляции.

Теперь мы можем менять наш исходный код и изменения автоматически будут отображаться в браузере.

Вернемся к файлам нашего приложения в VS Code.

Начнем с файла *package.json*. Он содержит настройки нашего приложения. Некоторые фрагменты из него:

```
{  
  "name": "hello-react",  
  "version": "0.1.0",
```

- **name** — имя нашего приложения,
- **version** — версия нашего приложения.

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
```

Секция `scripts` описывает действия при получении той или иной команды. Мы уже использовали `start` (эта команда запускает `development` версию вашего приложения).

Если у вас всё готово для использования приложения клиентом, тогда вам нужно получить оптимизированную сборку вашего приложения. Для этого нужно выполнить

```
npm run build
```

Для тестирования приложения нужно выполнить

```
npm test
```

Внимательно ознакомьтесь со структурой файла *package.json*.

Перейдем к другим файлам нашего проекта.

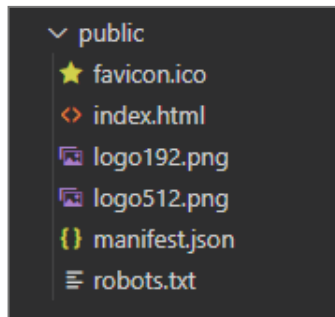


Рисунок 52

Папка *public* содержит набор иконок и других файлов. В нашем первом проекте большая часть этих файлов не нужна. Оставим только *index.html*. Изменим его содержимое:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Our First App</title>
  </head>
  <body>
    <noscript>
      You need to enable JavaScript to run this app.
    </noscript>
    <div id="root"></div>
  </body>
</html>
```

Папка *src* содержит файлы с кодом нашего приложения.

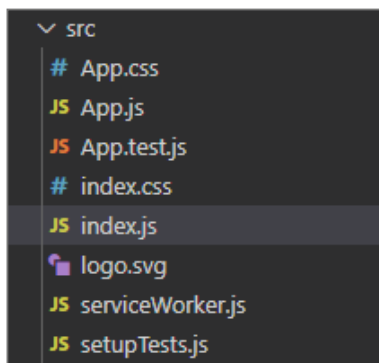


Рисунок 53

Файл *App.js* содержит *App* компонент нашего приложения. Файл *index.js* — точка входа в наше приложение.

Удалим все файлы в этой папке, кроме *App.js* и *index.js*.
Изменим исходный код в *App.js* на наш код:

```
import React from "react";

function App() {
  return <h1> Hello World! </h1>;
}

export default App;
```

Изменим исходный код в *index.js* на наш код:

```
import React from "react";
import ReactDOM from "react-dom";
import App from "../App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

Вот все файлы, которые мы оставили в папках *src* и *public*

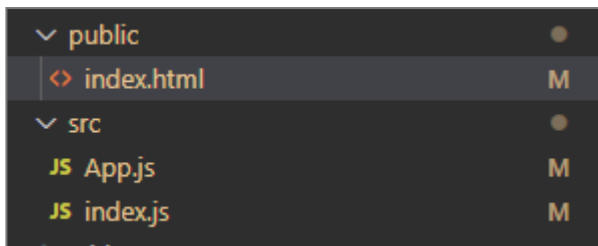


Рисунок 54

Пересоберем наше приложение. Посмотрим на наш результат в браузере.



Рисунок 55

Всё работает 😊. Ура!

Создайте свою копию базового приложения. Поэкспериментируйте с ним. Мы уверены, что вы преодолеете все трудности и запустите его локально на вашем компьютере.

Домашнее задание

- Пользователь вводит в два текстовых поля начало и конец диапазона, количество чисел. Приложение генерирует набор случайных чисел на основании введенных данных. Используйте API сайта *random.org*. Подробнее об использовании API читайте тут <https://api.random.org/>.
- Реализуйте приложение «Орел или Решка». Пользователь загадывает орла или решку. Приложение случайным образом выбирает орла или решку. Если выбор пользователя совпал с результатом, он выиграл, иначе проиграл. Используйте API сайта *random.org*. Подробнее об использовании API читайте тут <https://api.random.org/>.
- Реализуйте приложение «Игральные кости». Компьютер и пользователь по очереди бросают кости. Побеждает тот, кто выбросит большую сумму. Реализовать интерфейс для запуска новой игры. При броске нужно отображать игральные кости с количеством очков на каждом кубике. Используйте API сайта *random.org*. Подробнее об использовании API читайте тут <https://api.random.org/>.